PERTEMUAN 2 · Teori terapan · Kuis konsep

Teori Bahasa Formal dan Hierarki Chomsky

Universitas Bale Bandung

Fakultas Teknologi Informasi · Program Studi Teknik Informatika

Mata Kuliah Teknik Kompilasi (MWCA017) · Semester Ganjil · 3 SKS

Dosen: Cecep Suwanda, S.Si., M.Kom.

Capaian Pertemuan

CPMK induk: CPMK 01 — Menjelaskan tahapan kompilasi (*lexical*, *syntax*, *semantic analysis*).

Sub-CPMK yang dinilai pada pertemuan ini

Sub-CPMK 1.4: Membuat tabel pemetaan hierarki Chomsky (Tipe 3–0) ke jenis automata dan menguji klasifikasi lima contoh grammar ke kelas yang tepat.

Luaran yang diperiksa

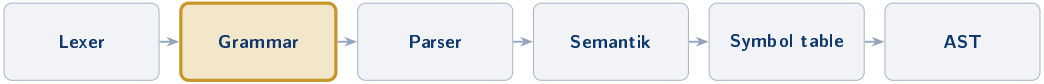
Tabel empat baris yang lengkap, klasifikasi lima grammar beserta alasan berbasis bentuk aturan, dan hasil uji string anggota serta bukan anggota.

Agenda Pertemuan (150 menit)

Menit	Kegiatan
10	Pembuka dan orientasi capaian
35	Alfabet, string, bahasa, serta tata bahasa dan derivasi
40	Hierarki Chomsky Tipe 3–0 dan automata pengenalnya
25	Tabel pemetaan dan klasifikasi lima grammar
20	Kuis konsep dan pembahasan
20	Diskusi implikasi bagi desain lexer dan parser

Pertemuan teori terapan: ceramah singkat, lokakarya tabel, lalu kuis konsep.

Posisi Pertemuan pada Rantai Proyek



- Pertemuan ini menyediakan landasan formal untuk simpul *grammar* pada rantai proyek.
- Teori bahasa menjelaskan mengapa lexer cukup memakai otomata hingga dan parser memerlukan otomata pushdown.
- Pertemuan ini tidak menambah increment kode; keluarannya berupa tabel dan uji klasifikasi.

Alfabet, String, Bahasa, dan Operasinya

Definisi dasar

Alfabet Σ adalah himpunan berhingga dan tidak kosong dari simbol. String adalah barisan berhingga simbol; panjangnya $|w|$, dan string kosong ε panjangnya nol. Himpunan seluruh string adalah Σ^*. Bahasa $L \subseteq \Sigma^*$ adalah himpunan string.

Operasi bahasa

- gabungan: $L_1 \cup L_2$;
- konkatenasi: $L_1 L_2 = \{xy \mid x \in L_1, y \in L_2\}$;
- bintang Kleene: $L^* = \bigcup_{k \geq 0} L^k$;
- *positive closure*: $L^+ = \bigcup_{k \geq 1} L^k$.

Contoh

$\Sigma = \{a, b\}$, $L = \{a, bb\}$.
L^* memuat $\varepsilon, a, bb, aa, abb, bba, bbbb, \ldots$; L^+ sama tanpa ε.

Bahasa sebagai Masalah Keputusan

- Setiap bahasa L mendefinisikan masalah keputusan: diberikan $w \in \Sigma^*$, tentukan apakah $w \in L$.
- *Lexer* menjawab masalah ini untuk bahasa token.
- Parser menjawabnya untuk bahasa program.

Kelas bahasa ditentukan oleh mesin pengenal

Kelas bahasa	Mesin yang memutuskan
Regular	Otomata hingga (DFA/NFA)
Bebas konteks	Otomata pushdown (PDA)
Sensitif konteks	Otomata terbatas linear
Rekursif terbilang	Mesin Turing

Tata Bahasa sebagai Empat-Tupel

Definisi

$G = (V, T, P, S)$, dengan V himpunan nonterminal, T himpunan terminal, P himpunan aturan produksi, dan $S \in V$ simbol awal. Himpunan V dan T saling lepas.

- Aturan berbentuk $\alpha \to \beta$, dengan α memuat paling sedikit satu nonterminal.
- Jika $\alpha = A \in V$, aturan disebut bebas konteks.
- Bahasa yang dibangkitkan: $L(G) = \{w \in T^* \mid S \Rightarrow^* w\}$.
- Dua tata bahasa ekivalen jika $L(G_1) = L(G_2)$.

Contoh: $S \to AB$, $A \to aA \mid a$, $B \to bB \mid b$.

Derivasi dan Bentuk Kalimat

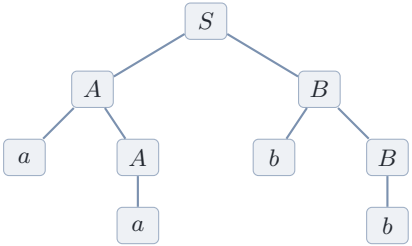
- Satu langkah: $\alpha A\beta \Rightarrow \alpha\gamma\beta$ jika $A \to \gamma \in P$.
- $\Rightarrow^*$ berarti nol atau lebih langkah; $\Rightarrow^+$ berarti satu atau lebih langkah.
- Bentuk kalimat adalah string yang dapat diturunkan dari S.

Contoh: string $aabb$ dari $S \to AB,\ A \to aA \mid a,\ B \to bB \mid b$

Leftmost: $S \Rightarrow AB \Rightarrow aAB \Rightarrow aaB \Rightarrow aabB \Rightarrow aabb$.
Rightmost: $S \Rightarrow AB \Rightarrow AbB \Rightarrow Abb \Rightarrow aAbb \Rightarrow aabb$.

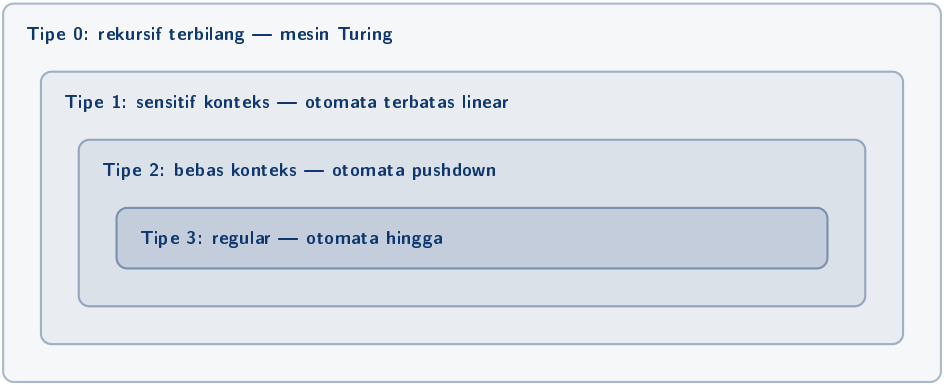
Keduanya menghasilkan pohon urai yang sama karena tata bahasa ini tidak ambigu.

Pohon Urai



- Akar: simbol awal S.
- Simpul dalam: nonterminal.
- Daun: terminal; dibaca kiri ke kanan menghasilkan $aabb$.
- Satu string dapat memiliki dua pohon berbeda jika tata bahasa ambigu; hal ini dibahas pada Pertemuan 7.

Hierarki Chomsky: Empat Kelas Bersarang



Semakin longgar bentuk aturan, semakin besar daya ekspresif kelas bahasa; setiap kelas termuat dalam kelas di atasnya.

Tipe 3: Tata Bahasa Regular

Bentuk aturan

$A \to aB$, $A \to a$, atau $A \to \varepsilon$, dengan $A, B \in V$ dan $a \in T$ (*right-linear*). Bentuk *left-linear* $A \to Ba$ menghasilkan kelas bahasa yang sama.

- Ekivalen dengan ekspresi reguler dan otomata hingga.
- Cocok untuk token: identifier, bilangan, operator.
- Contoh bahasa: a^*b^* dan himpunan identifier.

Identifier: huruf/garis bawah, lalu huruf/angka

$I \to hI \mid _I \mid h, \quad R \to hR \mid dR \mid \varepsilon,$
dengan h huruf, d angka, dan R sisa identifier.

Tipe 2: Tata Bahasa Bebas Konteks

Bentuk aturan

$A \to \alpha$, dengan $A \in V$ dan $\alpha \in (V \cup T)^*$. Kelas ini dikenali oleh otomata pushdown (*pushdown automaton*, PDA).

- Mampu menangani struktur bersarang: kurung, blok, pemanggilan.
- Contoh: $S \to aSb \mid \varepsilon$ membangkitkan $\{a^n b^n \mid n \geq 0\}$.
- Contoh ekspresi: $E \to E + T \mid T$, $T \to T * F \mid F$, $F \to (E) \mid \texttt{id}$.
- Tertutup terhadap gabungan, konkatenasi, dan bintang Kleene; tidak tertutup terhadap irisan dan komplemen.

Tipe 1: Tata Bahasa Sensitif Konteks

Bentuk aturan

$\alpha A \beta \to \alpha \gamma \beta$, dengan $\gamma \neq \varepsilon$. Nonterminal A diganti hanya dalam konteks α dan β, dan panjang string tidak boleh berkurang.

- Dikenali oleh otomata terbatas linear (*linear bounded automaton*).
- Contoh: $\{a^n b^n c^n \mid n \geq 1\}$, yang memerlukan pencocokan tiga kelompok berjumlah sama.
- Satu stack tidak cukup; diperlukan batas pita sepanjang masukan.

Tipe 0: Tata Bahasa Tanpa Batas

Bentuk aturan

$\alpha \to \beta$ tanpa batasan bentuk, asalkan α memuat paling sedikit satu nonterminal.

- Ekivalen dengan mesin Turing.
- Mencakup bahasa yang dapat dikenali secara rekursif terbilang.
- Sebagian bahasa pada kelas ini tidak dapat diputuskan.
- Contoh klasik: *halting problem* — tidak ada algoritma yang menentukan apakah program sembarang berhenti untuk semua masukan.

Tabel Pemetaan Hierarki Chomsky (Artefak Sub-CPMK 1.4)

Tipe	Kelas bahasa	Bentuk aturan	Automata	Contoh
3	Regular	$A \to aB$, $A \to a$	DFA/NFA	a^*b^*
2	Bebas konteks	$A \to \alpha$	PDA	$\{a^n b^n\}$
1	Sensitif konteks	$\alpha A \beta \to \alpha \gamma \beta$	Otomata terbatas linear	$\{a^n b^n c^n\}$
0	Rekursif terbilang	$\alpha \to \beta$	Mesin Turing	bahasa yang dikenali mesin Turing

Setiap baris harus konsisten: contoh bahasa pada kolom terakhir benar-benar dikenali oleh automata pada kolom keempat. Tabel inilah artefak Sub-CPMK 1.4.

Contoh Berpenyelesaian: Klasifikasi Lima Grammar

	Aturan produksi	Tipe	Alasan
(a)	$S \to aS \mid bS \mid \varepsilon$	3	Semua aturan berbentuk $A \to aB$ atau $A \to \varepsilon$.
(b)	$S \to aSb \mid \varepsilon$	2	Aturan $A \to \alpha$; membangkitkan $\{a^n b^n\}$.
(c)	$S \to aSBC \mid aBC$, $CB \to BC$, $aB \to ab$, $bB \to bb$, $bC \to bc$, $cC \to cc$	1	Aturan $CB \to BC$ bergantung konteks; panjang tidak berkurang; membangkitkan $\{a^n b^n c^n\}$.
(d)	$S \to AB$, $AB \to BA$, $A \to a$, $B \to b$	0	$AB \to BA$ tidak memenuhi batasan sensitif konteks.
(e)	$S \to Sa \mid b$	3	Aturan $A \to Ba$, yaitu *left-linear*.

Uji tambahan: setiap grammar diuji dengan dua string anggota dan satu string bukan anggota.

Implikasi bagi Desain Lexer dan Parser

Fase	Formalisme	Alasan
Leksikal	Regular/DFA	Struktur token sederhana dan efisien
Sintaksis	CFG/PDA	Struktur bersarang memerlukan stack
Semantik	*Attribute grammar*	Memerlukan konteks dan tipe
Validasi	Aturan dan traversal	Memerlukan analisis lintas simpul

Peringatan desain

Jangan memakai ekspresi reguler untuk memeriksa struktur bersarang seperti kurung seimbang. Ekspresi reguler tidak memiliki memori tak terbatas; gunakan parser atau PDA.

Kuis Konsep

1. Apa perbedaan string dan bahasa?
2. Tata bahasa $S \to aS \mid bS \mid \varepsilon$ termasuk Tipe berapa? Sebutkan alasannya.
3. Mengapa $\{a^n b^n \mid n \geq 0\}$ bukan bahasa regular? Mesin apa yang mengenalinya?
4. Automata apa yang mengenali bahasa sensitif konteks?
5. Mengapa lexer cukup memakai Tipe 3, sedangkan parser memerlukan Tipe 2?

Kerjakan mandiri selama 8 menit, lalu bahas bersama. Kunci tersedia pada versi handout.

Kegiatan Mahasiswa di Kelas

Lokakarya: tabel pemetaan dan uji lima grammar

1. Lengkapi tabel Tipe 3–0: kelas bahasa, bentuk aturan, automata, dan contoh bahasa.
2. Tentukan tipe lima grammar yang diberikan beserta alasannya.
3. Uji setiap grammar dengan minimal tiga string: dua anggota dan satu bukan anggota.
4. Cocokkan hasil uji dengan automata yang dipetakan.
5. Kumpulkan tabel, klasifikasi, dan hasil uji.

Bekerja berpasangan; satu pasangan memeriksa tabel pasangan lain sebelum dikumpulkan.

Tugas dan Artefak

Artefak Sub-CPMK 1.4

Tabel pemetaan hierarki Chomsky Tipe 3–0 ke jenis automata, disertai klasifikasi lima contoh grammar dan hasil uji string.

- Tabel empat baris lengkap: tipe, kelas bahasa, bentuk aturan, automata, dan contoh bahasa.
- Lima grammar terklasifikasi dengan alasan berbasis bentuk aturan.
- Setiap grammar diuji dengan dua string anggota dan satu string bukan anggota.
- Tidak ada contoh bahasa yang salah kelas.

Artefak ini menjadi bukti Sub-CPMK 1.4 pada Kuis Konsep (3%) dan UTS Praktik (14%).

Rangkuman dan Jembatan

- Alfabet, string, dan bahasa didefinisikan secara formal; bahasa adalah himpunan string atas alfabet.
- Tata bahasa $G = (V, T, P, S)$ membangkitkan bahasa melalui derivasi; leftmost dan rightmost dapat menghasilkan pohon yang sama.
- Hierarki Chomsky bersarang: Tipe 3 regular, Tipe 2 bebas konteks, Tipe 1 sensitif konteks, Tipe 0 tanpa batas.
- Setiap tipe bersesuaian dengan automata: DFA/NFA, PDA, otomata terbatas linear, dan mesin Turing.
- Pilihan formalisme mengikuti struktur masalah: token memakai Tipe 3, sintaksis memakai Tipe 2, semantik melampaui Tipe 2.

Pertemuan berikutnya

Pertemuan 3 memulai analisis leksikal: token, lexeme, pattern, dan ekspresi reguler. Teori Tipe 3 berubah menjadi tabel ekspresi reguler untuk setiap kategori token subset bahasa C.